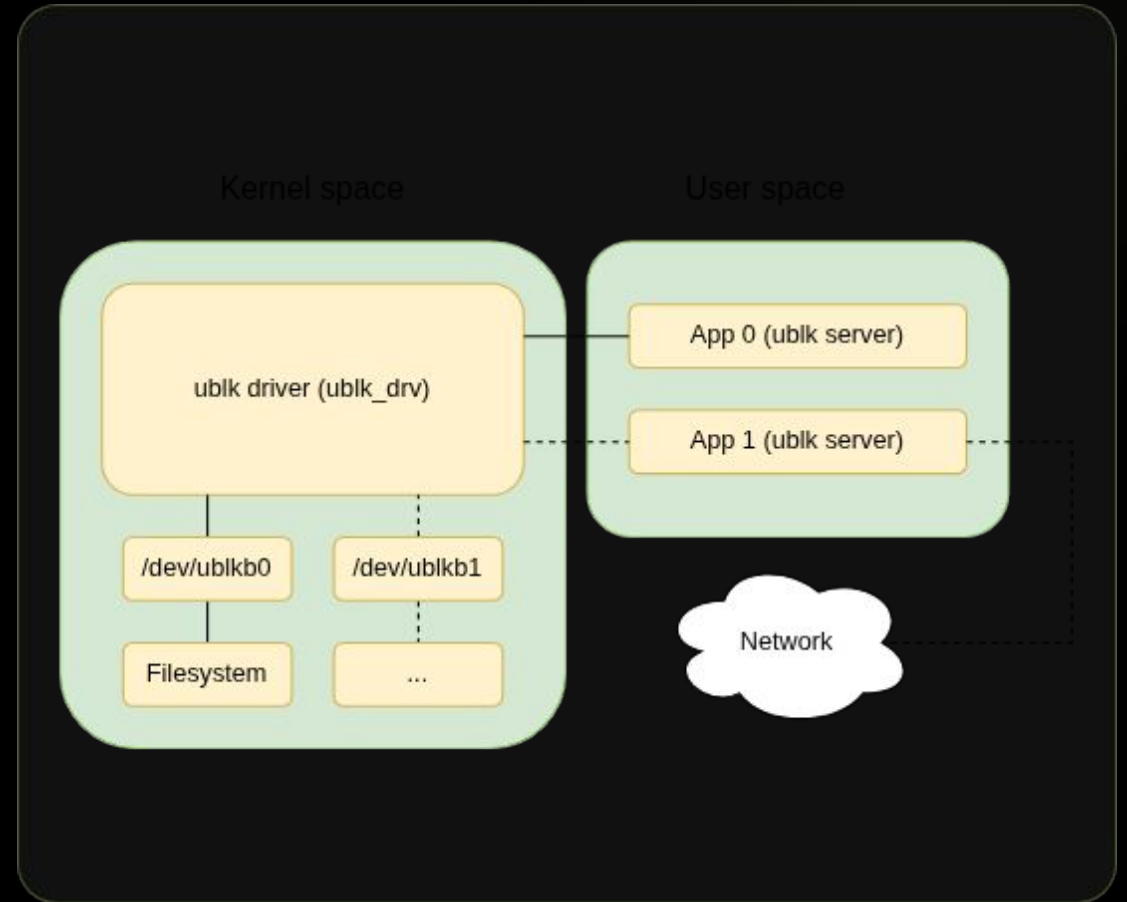


ublk: use cases and progress

Ming Lei CLSF2026

UBLK: Introduction

- › Block device in userspace
- › IO-uring based
- › High performance, easy to implement & use, flexible
- › Merged to v6.0



UBLK: Use cases

> Android OTA (snapshot virtual device)

https://cs.android.com/android/platform/superproject/+/android-latest-release:system/core/fs_mgr/libsnapshot/snapuserd/ublk_block_server.cpp

ublk based zero copy I/O - use case in Android: LPC2024

> Longhorn

Distributed storage for K8s

UBLK Frontend Support for SUSE Storage (Longhorn) v2 Data Engine

<https://www.suse.com/c/ublk-frontend-support-for-suse-storage-longhorn-v2-data-engine>

<https://github.com/longhorn/longhorn/wiki/Longhorn-Performance-Investigation>

UBLK + v1 engine: **4X** IOPS of ISCSI + v1 volume

UBLK + v2 engine: IOPS **+37%** than NVMeOF + v2 volume

Optimizing the Longhorn Cloud-native Software Defined Storage Engine for High Performance

<https://arxiv.org/html/2502.14419v1>

UBLK: Use cases

> **Hyperscale Flash Storage in Pure storage**

SNIA SDC 2025 - Optimizing Hyperscale Flash Storage: Efficient QLC, Infinite Scale
<https://www.snia.org/sniadeveloper/session/19357>

host-based FTL running in userspace / QLC Nand

24 DirectFlash Module (DFM) in 2U server

RAID

Expose block device via UBLK

UBLK: Use cases

> SPDK

<https://spdk.io/doc/ublk.html>

> ublkpp: one raid library

ublkpp — a C++23 RAID library built on ublksrv, used in production <https://github.com/szmyd/ublkpp>
<https://github.com/ublk-org/ublksrv/discussions/183>

> Multikernel: Isolating Storage Queues With ublk (LPC2025)

<https://lpc.events/event/19/contributions/2074/attachments/1776/3957/Multikernel-LPC2025.pdf>

> SIMD-accelerated LZ4/ZSTD compression with ublk block device for transparent memory compression

<https://github.com/paiml/aprender/tree/main/crates/aprender-zram>

UBLK: Use cases

> Other private use cases

Deepseek, Nvidia, Alibaba, Sony playstation,

> Other scholar use cases

PageFlex: Flexible and Efficient User-space Delegation of Linux Paging Policies with eBPF
<https://www.usenix.org/system/files/atc25-yelam.pdf>

AutoSSD: CXL-Enhanced Autonomous SSDs for Low Tail Latency
<<https://dl.acm.org/doi/pdf/10.1145/3731545.3731579>>

Encryption for userspace-managed block devices in Linux <https://share.google/yU21ZnbEyWMcroaRC>

....

UBLK: Progress

> Zero copy

Jon Corbet, "Zero-copy I/O for ublk, three different ways," [LWN.net](https://lwn.net/Articles/926118/) <https://lwn.net/Articles/926118/>

27cb27b6d5ea io_uring: add support for kernel registered bvecs (v6.15)

1045afae4b88 io_uring: support vectored kernel fixed buffer (v6.15)

99c1e4eb6a3f ublk: register buffer to local io_uring with provided buf index via UBLK_F_AUTO_BUF_REG (v6.16)

fuse/io-uring: add kernel-managed buffer rings and zero-copy: <https://lwn.net/Articles/1049130/>

zero copy is done in physical page level, no user mapping available

[PATCH v3 0/12] io_uring: add IORING_OP_BPF for extending io_uring <https://lore.kernel.org/io-uring/20260324163753.1900977-1-ming.lei@redhat.com/>

[LSF/MM/BPF TOPIC] User space RAID5 with ublk and io_uring BPF

<https://lore.kernel.org/linux-block/CALTww28QMg=YXqKWpWLZrLO+xiqOe3LGyput8dx68-dnQsxdg@mail.gmail.com/>

UBLK: Progress

> **shared memory based Zero copy**

08677040a911 ublk: enable UBLK_F_SHMEM_ZC feature flag

cooperation based zero copy by sharing memory between ublk frontend and backend(memfd, hugetlb, ...), DIRECTIO only

Shared user virtual mapping is available

IO buffer is often pre-allocated & long lived, so runtime DMA mapping can be saved

ublk/nvme_vfio: <https://github.com/ublk-org/ublkdrv/tree/master/targets/nvme> (customized nvme driver)

[PATCH v3 00/10] Add dmabuf read/write via io_uring
<https://lore.kernel.org/io-uring/cover.1777475843.git.asml.silence@gmail.com/>

Axboe Hacking On New Linux Patches For 60% Increase To Per-Core I/O Performance
<https://www.phoronix.com/news/Axboe-60p-Faster-Per-Core-Perf>
https://git.kernel.org/pub/scm/linux/kernel/git/axboe/linux.git/log/?h=io_uring-io-slots

UBLK: Progress

> UBLK_F_BPF (in-progress)

Submit IO from bpf struct_ops prog in frontend context (nvme vfio)

Reduce submission latency by avoiding roundtrip

> block IO integrity

b2503e936b59 ublk: support UBLK_F_INTEGRITY

> UBLK_F_BATCH_IO

e2723e6ce602 ublk: add new feature UBLK_F_BATCH_IO

> syzkaller support

sys/linux: add syzkaller definitions to fuzz ublk driver <https://github.com/google/syzkaller/pull/7039>

Questions?